CUDA C++ Introduction - GPU 프로그래밍 시작하기

학습 목표

- GPU와 CPU의 차이점 이해
- CUDA 프로그래밍 모델의 기본 개념 파악
- CUDA 개발 환경 설정
- 첫 번째 CUDA 프로그램 작성 및 실행

예상 학습 시간: 3-4시간
난이도: ⭐ (입문)

---

1. GPU vs CPU: 왜 GPU가 필요한가?

1.1 CPU의 설계 철학: 순차 처리의 달인

```
CPU (Intel Core i9-13900K 예시)
┌──────────────────────────────────────┐
│  Complex Control Logic               │  ← 분기 예측, 비순차 실행
│  ┌────┐ ┌────┐ ┌────┐ ┌────┐        │
│  │Core│ │Core│ │Core│ │Core│        │  ← 24코어 (8P + 16E)
│  └────┘ └────┘ └────┘ └────┘        │
│  Large L1/L2/L3 Cache (36MB)         │  ← 캐시 메모리 크다
│  High Clock Speed (5.8 GHz)          │  ← 클럭 속도 높다
└──────────────────────────────────────┘

강점: 복잡한 제어 흐름, 낮은 지연시간
약점: 병렬 처리 능력 제한적 (최대 ~50 threads)
```

1.2 GPU의 설계 철학: 대규모 병렬 처리

```
GPU (NVIDIA RTX 4090 예시)
┌──────────────────────────────────────────┐
│  Simple Control (per SM)                 │  ← 간단한 제어 로직
│  ┌──────┐ ┌──────┐ ┌──────┐     ┌──────┐  │
│  | SM   | | SM   | | SM   | ... | SM   |  │  ← 128 SMs
│  |128   | |128   | |128   |     |128   |  │
│  |cores| |cores| |cores|     |cores|  │  ← SM당 128 CUDA cores
│  └──────┘ └──────┘ └──────┘     └──────┘  │
│  Small Cache, Lower Clock (2.5 GHz)      │  ← 캐시 작고 클럭 낮다
│  Total: 16,384 CUDA cores                │  ← 하지만 코어가 엄청 많다!
└──────────────────────────────────────────┘

강점: 대규모 병렬 처리 (수만 threads 동시 실행)
약점: 복잡한 제어 흐름에 약함, 지연시간 높음
```

1.3 언제 GPU를 사용해야 하는가?

✅ GPU에 적합한 작업:

```
// 예시 1: 벡터 덧셈 (100% 병렬 가능)
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];  // 각 iteration이 독립적!
}
// N = 1,000,000이면 → 1,000,000개 threads로 동시 실행 가능

// 예시 2: 이미지 처리 (픽셀별 독립 연산)
for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        pixel[y][x] = blur(pixel[y][x]);  // 각 픽셀 독립!
    }
}
// 1920x1080 = 2,073,600 픽셀 → 200만 threads로 동시 실행
```

핵심 특징: 데이터 병렬성 (Data Parallelism)

- 같은 연산을 다른 데이터에 반복 적용
- Iteration들이 서로 독립적
- SIMD (Single Instruction, Multiple Data) 패턴

❌ GPU에 부적합한 작업:

```
// 예시 1: 순차적 의존성 (Fibonacci)
for (int i = 2; i < N; i++) {
    fib[i] = fib[i-1] + fib[i-2];  // i번째가 (i-1), (i-2)에 의존!
}
// 병렬화 불가능 - 순차 실행 필요

// 예시 2: 복잡한 분기 (파싱)
while (!eof) {
    char c = getchar();
    if (c == '{') { /* 복잡한 처리 */ }
    else if (c == '[') { /* 다른 복잡한 처리 */ }
    // ... 수십 개의 분기
}
// Thread마다 다른 경로 → Warp divergence (성능 저하)
```

1.4 성능 비교: CPU vs GPU

벡터 덧셈 (N = 100M 원소) 벤치마크:

```
CPU (16 threads, Intel i9-13900K):
  Time: ~120ms
  Throughput: 833 million ops/sec

GPU (NVIDIA RTX 4090):
  Time: ~2ms
  Throughput: 50,000 million ops/sec

Speedup: 60x faster! 🚀
```

하지만 주의! GPU는 데이터 전송 오버헤드가 있습니다:

```
CPU → GPU 메모리 복사: ~10ms (100M 원소 기준)
GPU 계산: ~2ms
GPU → CPU 메모리 복사: ~10ms
──────────────────────────────────────
Total: ~22ms (실제 speedup: 5.4x)
```

교훈: 계산량이 충분히 클 때만 GPU가 유리합니다!

---

2. CUDA 프로그래밍 모델 개요

2.1 핵심 개념 3가지

(1) Kernel: GPU에서 실행되는 함수

```
__global__ void vectorAdd(float* A, float* B, float* C, int N) {
    int i = threadIdx.x;  // 이 thread의 인덱스
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

- `__global__` : GPU에서 실행, CPU에서 호출
- 수천~수만 threads가 **동시에** 이 함수 실행

(2) Thread Hierarchy: Grid → Block → Thread

```
Grid (전체 작업)
  ├─ Block 0 (1024 threads)
  |    ├─ Thread 0
  |    ├─ Thread 1
  |    └─ ...
  ├─ Block 1 (1024 threads)
  └─ Block 2 (1024 threads)
```

(3) Memory Hierarchy: 여러 종류의 메모리

```
Register (fastest)  ← thread마다 개인 공간
    ↓
Shared Memory       ← block 내부 threads 공유
    ↓
Global Memory       ← 모든 threads 공유 (slowest)
```

2.2 CUDA 실행 흐름

```
┌──────────┐                          ┌──────────┐
|   CPU    |                          |   GPU    |
|  (Host)  |                          | (Device) |
└──────────┘                          └──────────┘
     |
     | 1. 데이터 준비 (Host memory)
     |    float A[N], B[N], C[N];
```

```
│
├─ 2. GPU 메모리 할당
│    cudaMalloc(&d_A, size);
│    ──────────────────────────────▶ [GPU Memory]
│
├─ 3. Host → Device 복사
│    cudaMemcpy(d_A, A, ...);
│    ═══════════════════════════════▶ [d_A, d_B]
│
├─ 4. Kernel 실행
│    vectorAdd<<<grid, block>>>(d_A, d_B, d_C, N);
│                                    │
│                                    ├─ Thread 0: C[0] = A[0] + B[0]
│                                    ├─ Thread 1: C[1] = A[1] + B[1]
│                                    ├─ Thread 2: C[2] = A[2] + B[2]
│                                    └─ ... (병렬 실행!)
│
├─ 5. Device → Host 복사
│    cudaMemcpy(C, d_C, ...);
│    ◀═══════════════════════════════ [d_C]
│
└─ 6. GPU 메모리 해제
     cudaFree(d_A);
     ◀────────────────────────────── [Free]
```

---

3. CUDA 개발 환경 설정

3.1 필요 사항

하드웨어:

- **NVIDIA GPU** (CUDA Compute Capability 3.0 이상)
  - GeForce: GTX 700 시리즈 이상
  - Quadro: K-series 이상
  - Tesla/Data Center: K40 이상
  - 확인 방법: https://developer.nvidia.com/cuda-gpus

소프트웨어:

- **CUDA Toolkit** (최신 버전 권장, 현재 12.3)
- **호환되는 NVIDIA Driver**
- **C/C++ 컴파일러**:
  - Linux: GCC 7.x - 12.x
  - Windows: Visual Studio 2019/2022

3.2 설치 (Ubuntu/Linux 예시)

Step 1: NVIDIA Driver 설치

```
# GPU 확인
lspci | grep -i nvidia

# Driver 설치 (자동)
sudo ubuntu-drivers autoinstall

# 또는 수동
sudo apt install nvidia-driver-535

# 재부팅 후 확인
nvidia-smi
```

Step 2: CUDA Toolkit 설치

```
# 다운로드 (https://developer.nvidia.com/cuda-downloads)
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/
sudo dpkg -i cuda-keyring_1.0-1_all.deb
sudo apt-get update
sudo apt-get install cuda

# 환경 변수 설정 (~/.bashrc에 추가)
export PATH=/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH

# 확인
nvcc --version
```

Step 3: 샘플 컴파일 테스트

```
# CUDA Samples 복사
cuda-install-samples-12.3.sh ~
cd ~/NVIDIA_CUDA-12.3_Samples/1_Utilities/deviceQuery

# 컴파일 및 실행
make
./deviceQuery
```

정상 출력 예시:

```
Device 0: "NVIDIA GeForce RTX 4090"
  CUDA Capability: 8.9
  Total Global Memory: 24217 MiB
  Multiprocessors: 128
  CUDA Cores: 16384
  ...
Result = PASS
```

3.3 첫 번째 CUDA 프로그램

hello_cuda.cu:

```
#include <stdio.h>
#include <cuda_runtime.h>

// GPU에서 실행될 Kernel 함수
__global__ void helloFromGPU() {
    printf("Hello from GPU thread %d in block %d!\n",
           threadIdx.x, blockIdx.x);
}

int main() {
    // CPU에서 실행
    printf("Hello from CPU!\n");

    // GPU Kernel 실행: 2개 block, 각 block에 4개 threads
    // 총 2 × 4 = 8개 threads 실행
    helloFromGPU<<<2, 4>>>();

    // GPU 작업 완료 대기
    cudaDeviceSynchronize();
```

```
    return 0;
}
```

컴파일 및 실행:

```
# NVCC로 컴파일
nvcc -o hello_cuda hello_cuda.cu

# 실행
./hello_cuda
```

출력:

```
Hello from CPU!
Hello from GPU thread 0 in block 0!
Hello from GPU thread 1 in block 0!
Hello from GPU thread 2 in block 0!
Hello from GPU thread 3 in block 0!
Hello from GPU thread 0 in block 1!
Hello from GPU thread 1 in block 1!
Hello from GPU thread 2 in block 1!
Hello from GPU thread 3 in block 1!
```

주의: Thread 출력 순서는 보장되지 않습니다! 병렬 실행이므로 매번 다를 수 있습니다.

---

4. CUDA 프로그래밍 기초 문법

4.1 Function Type Qualifiers

Qualifier	실행 위치	호출 위치	용도
`__global__`	Device (GPU)	Host (CPU)	Kernel 함수 (main entry point)
`__device__`	Device	Device	GPU 내부 함수
`__host__`	Host	Host	CPU 함수 (생략 가능)

```
// Kernel - CPU에서 호출, GPU에서 실행
__global__ void myKernel(int* data) {
    int result = helperFunction(data[threadIdx.x]);
}

// Device 함수 - GPU에서만 호출/실행
__device__ int helperFunction(int x) {
    return x * 2;
}

// Host 함수 - CPU에서만 호출/실행
__host__ void prepareData(int* data) {
    // CPU 코드
}

// 둘 다 컴파일 (CPU/GPU 버전 2개 생성)
__host__ __device__ int math_function(int x) {
    return x * x;
}
```

4.2 Kernel 실행 구문

```
kernelName<<<gridDim, blockDim, sharedMemSize, stream>>>(args...);
           └──────┬──────┘ └────┬────┘ └──────┬──────┘
                  |            |              |
                  |            |              └─ 고급 기능 (나중에 학습)
                  |            └─ Block당 thread 개수
                  └─ Grid의 block 개수
```

예시:

```
// 1D 구성
myKernel<<<10, 256>>>(d_data);
// → 10 blocks × 256 threads/block = 2560 threads

// 2D 구성
dim3 grid(4, 3);    // 4×3 = 12 blocks
dim3 block(16, 16); // 16×16 = 256 threads/block
matrixKernel<<<grid, block>>>(d_matrix);
// → 12 blocks × 256 threads/block = 3072 threads

// 3D 구성
dim3 grid(8, 4, 2);    // 8×4×2 = 64 blocks
```

```
dim3 block(8, 8, 4);   // 8×8×4 = 512 threads/block
volumeKernel<<<grid, block>>>(d_volume);
// → 64 blocks × 512 threads/block = 32768 threads
```

4.3 Built-in Variables (Thread 식별)

```
__global__ void kernel() {
    // Grid 차원
    gridDim.x, gridDim.y, gridDim.z   // Grid의 block 개수

    // Block 차원
    blockDim.x, blockDim.y, blockDim.z // Block의 thread 개수

    // Block 인덱스
    blockIdx.x, blockIdx.y, blockIdx.z // 현재 block의 위치

    // Thread 인덱스
    threadIdx.x, threadIdx.y, threadIdx.z // Block 내 thread 위치
}
```

1D 예시: Global Thread ID 계산

```
__global__ void vectorAdd(float* A, float* B, float* C, int N) {
    // Global thread ID = block 시작 위치 + block 내 thread 위치
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < N) {  // 범위 체크 (N이 blockDim의 배수가 아닐 수 있음)
        C[tid] = A[tid] + B[tid];
    }
}

// 예시: N = 1000, blockDim = 256
// Block 0: threads 0-255
// Block 1: threads 256-511
// Block 2: threads 512-767
// Block 3: threads 768-1023 (하지만 1000-1023은 범위 밖)
```

2D 예시: 이미지 처리

```
__global__ void blurImage(float* input, float* output, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
```

```
    if (x < width && y < height) {
        int idx = y * width + x;  // 2D → 1D 변환
        output[idx] = computeBlur(input, x, y, width, height);
    }
}

// 실행 구성
dim3 block(16, 16);  // 256 threads per block
dim3 grid((width + 15) / 16, (height + 15) / 16);  // 올림 계산
blurImage<<<grid, block>>>(d_in, d_out, width, height);
```

4.4 Memory Management API

```
// 메모리 할당
cudaError_t cudaMalloc(void** devPtr, size_t size);

// 메모리 복사
cudaError_t cudaMemcpy(void* dst, const void* src, size_t count,
                       cudaMemcpyKind kind);
// kind: cudaMemcpyHostToDevice, cudaMemcpyDeviceToHost,
//       cudaMemcpyDeviceToDevice

// 메모리 해제
cudaError_t cudaFree(void* devPtr);

// 메모리 초기화
cudaError_t cudaMemset(void* devPtr, int value, size_t count);
```

전형적인 사용 패턴:

```
int N = 1024;
size_t bytes = N * sizeof(float);

// Host 메모리
float* h_A = (float*)malloc(bytes);
float* h_B = (float*)malloc(bytes);
float* h_C = (float*)malloc(bytes);

// Device 메모리
float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, bytes);
cudaMalloc(&d_B, bytes);
```

```
cudaMalloc(&d_C, bytes);

// Host → Device
cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

// Kernel 실행
vectorAdd<<<(N+255)/256, 256>>>(d_A, d_B, d_C, N);

// Device → Host
cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);

// 정리
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);
```

4.5 Error Handling

```
// 방법 1: 반환값 체크
cudaError_t err = cudaMalloc(&d_A, bytes);
if (err != cudaSuccess) {
    printf("CUDA Error: %s\n", cudaGetErrorString(err));
    exit(1);
}

// 방법 2: Helper Macro
#define CUDA_CHECK(call) \
do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA Error at %s:%d - %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(1); \
    } \
} while(0)

// 사용
CUDA_CHECK(cudaMalloc(&d_A, bytes));

// Kernel 에러 체크 (비동기 실행이므로 특별 처리)
myKernel<<<grid, block>>>(args);
CUDA_CHECK(cudaGetLastError());  // Launch error
CUDA_CHECK(cudaDeviceSynchronize());  // Execution error
```

5. 실습: Vector Addition

5.1 완전한 예제

examples/cuda/00_vector_add.cu:

```
#include <stdio.h>
#include <cuda_runtime.h>

// Error checking macro
#define CUDA_CHECK(call) \
do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA Error at %s:%d - %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(1); \
    } \
} while(0)

// Kernel: Vector Addition
__global__ void vectorAdd(const float* A, const float* B, float* C, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < N) {
        C[tid] = A[tid] + B[tid];
    }
}

int main() {
    // 1. 설정
    const int N = 1 << 20;  // 1M 원소
    size_t bytes = N * sizeof(float);

    printf("Vector Addition: %d elements (%.2f MB)\n", N, bytes / 1e6);

    // 2. Host 메모리 할당 및 초기화
    float* h_A = (float*)malloc(bytes);
    float* h_B = (float*)malloc(bytes);
    float* h_C = (float*)malloc(bytes);
```

```
for (int i = 0; i < N; i++) {
    h_A[i] = (float)i;
    h_B[i] = (float)(i * 2);
}

// 3. Device 메모리 할당
float *d_A, *d_B, *d_C;
CUDA_CHECK(cudaMalloc(&d_A, bytes));
CUDA_CHECK(cudaMalloc(&d_B, bytes));
CUDA_CHECK(cudaMalloc(&d_C, bytes));

// 4. Host → Device 복사
CUDA_CHECK(cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice));

// 5. Kernel 실행 구성
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

printf("Launching kernel: %d blocks × %d threads = %d total threads\n",
       blocksPerGrid, threadsPerBlock, blocksPerGrid * threadsPerBlock);

// 6. Kernel 실행
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());

// 7. Device → Host 복사
CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));

// 8. 결과 검증
bool success = true;
for (int i = 0; i < N; i++) {
    float expected = h_A[i] + h_B[i];
    if (fabs(h_C[i] - expected) > 1e-5) {
        printf("Error at index %d: %f != %f\n", i, h_C[i], expected);
        success = false;
        break;
    }
}

if (success) {
    printf("✓ Verification PASSED!\n");
}

// 9. 정리
```

```
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);

    return 0;
}
```

5.2 컴파일 및 실행

```
# 기본 컴파일
nvcc -o vector_add 00_vector_add.cu

# 최적화 옵션
nvcc -O3 -o vector_add 00_vector_add.cu

# 아키텍처 지정 (RTX 4090 = sm_89)
nvcc -arch=sm_89 -O3 -o vector_add 00_vector_add.cu

# 실행
./vector_add
```

출력:

```
Vector Addition: 1048576 elements (4.19 MB)
Launching kernel: 4096 blocks × 256 threads = 1048576 total threads
✓ Verification PASSED!
```

5.3 성능 측정

CPU 버전과 비교:

```
// CPU 버전
#include <chrono>

auto start = std::chrono::high_resolution_clock::now();
for (int i = 0; i < N; i++) {
    h_C[i] = h_A[i] + h_B[i];
}
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - star
printf("CPU Time: %.3f ms\n", duration.count() / 1000.0);
```

GPU 버전 (CUDA Events 사용):

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaEventRecord(stop);

cudaEventSynchronize(stop);
float milliseconds = 0;
cudaEventElapsedTime(&milliseconds, start, stop);
printf("GPU Kernel Time: %.3f ms\n", milliseconds);

cudaEventDestroy(start);
cudaEventDestroy(stop);
```

예상 결과 (RTX 4090):

```
CPU Time (single thread): ~15 ms
GPU Kernel Time: ~0.5 ms
Speedup: 30x
```

---

6. NVCC 컴파일러 옵션

6.1 기본 옵션

```
# 기본 컴파일
nvcc -o output input.cu

# 최적화 레벨
nvcc -O0  # 최적화 없음 (디버깅용)
nvcc -O2  # 중간 최적화
nvcc -O3  # 최대 최적화 (권장)

# 디버그 정보 포함
nvcc -g -G -o output input.cu
# -g: Host 디버그 정보
# -G: Device 디버그 정보 (성능 저하!)
```

6.2 아키텍처 지정

```
# Compute Capability 지정
nvcc -arch=sm_80 input.cu  # Ampere (A100, RTX 30xx)
nvcc -arch=sm_89 input.cu  # Ada Lovelace (RTX 40xx)
nvcc -arch=sm_90 input.cu  # Hopper (H100)

# Virtual Architecture (PTX) + Real Architecture
nvcc -gencode arch=compute_80,code=sm_80 input.cu

# 여러 아키텍처 동시 컴파일 (fat binary)
nvcc -gencode arch=compute_80,code=sm_80 \
     -gencode arch=compute_89,code=sm_89 \
     input.cu
```

주의: 자신의 GPU Compute Capability를 확인하세요!

```
# deviceQuery 실행
./deviceQuery | grep "CUDA Capability"
# 출력: CUDA Capability Major/Minor version number: 8.9
# → sm_89 사용
```

6.3 유용한 옵션

```
# 경고 활성화
nvcc -Xcompiler -Wall -Xcompiler -Wextra input.cu

# CUDA 버전 지정
nvcc -std=c++17 input.cu

# PTX 코드 생성 (디버깅/분석용)
nvcc -ptx input.cu  # → input.ptx 생성

# SASS (어셈블리) 코드 보기
nvcc -cubin input.cu  # → input.cubin 생성
cuobjdump -sass input.cubin

# 레지스터 사용량 출력
nvcc --ptxas-options=-v input.cu
```

---

7. 디버깅 및 문제 해결

7.1 흔한 에러들

에러 1: `identifier "threadIdx" is undefined`

```
// 잘못된 코드
void myFunction() {
    int tid = threadIdx.x;  // ❌ __device__ 아닌 함수에서 사용
}

// 올바른 코드
__device__ void myFunction() {
    int tid = threadIdx.x;  // ✅ __device__ 함수
}
```

에러 2: `calling a __global__ function from a __global__ function`

```
// 잘못된 코드
__global__ void kernel1() { }
__global__ void kernel2() {
    kernel1<<<1, 1>>>();  // ❌ Kernel에서 Kernel 호출 불가
}

// 올바른 코드 (Dynamic Parallelism 필요 - 고급 기능)
__global__ void kernel1() { }
__global__ void kernel2() {
    // Dynamic Parallelism 활성화 필요
    // -rdc=true -lcudadevrt 컴파일 옵션
}
```

에러 3: `unspecified launch failure`

```
// 원인: 범위 벗어난 메모리 접근
__global__ void buggyKernel(float* data, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    data[tid] = 0.0f;  // ❌ tid >= N일 때 문제!
}

// 해결: 범위 체크
__global__ void fixedKernel(float* data, int N) {
```

```
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < N) {  // ✅ 범위 체크
        data[tid] = 0.0f;
    }
}
```

7.2 디버깅 도구

cuda-gdb (Linux):

```
# 디버그 정보 포함하여 컴파일
nvcc -g -G -o program program.cu

# cuda-gdb 실행
cuda-gdb ./program

# 주요 명령어
(cuda-gdb) break myKernel  # Kernel에 breakpoint
(cuda-gdb) run
(cuda-gdb) cuda thread     # 현재 CUDA thread 정보
(cuda-gdb) cuda block      # 현재 CUDA block 정보
(cuda-gdb) print threadIdx.x
```

compute-sanitizer (메모리 에러 검출):

```
# 실행
compute-sanitizer ./program

# 메모리 누수 체크
compute-sanitizer --tool memcheck ./program

# Race condition 체크
compute-sanitizer --tool racecheck ./program
```

---

8. 다음 단계

축하합니다! CUDA 기초를 마쳤습니다. 이제 다음 내용을 학습하세요:

1. [01_CUDA_Programming_Model.md]

    - Grid, Block, Thread 상세 분석
    - Warp 개념 도입
    - Occupancy 이해

2. **02_Memory_Hierarchy.md**

    - Global, Shared, Register 메모리 성능 비교
    - Memory Coalescing
    - Cache 활용

3. **실습 예제**:

    - `examples/cuda/01_matrix_multiply.cu` - 2D 인덱싱 연습
    - `examples/cuda/02_image_blur.cu` - 실전 이미지 처리

참고 자료

- CUDA C++ Programming Guide
- CUDA C++ Best Practices Guide
- CUDA Toolkit Documentation
- NVIDIA Developer Blog

학습 체크리스트:

- [ ] GPU와 CPU의 차이점 설명 가능
- [ ] CUDA Toolkit 설치 완료
- [ ] Hello World 프로그램 실행 성공
- [ ] Vector Addition 예제 이해 및 수정 가능
- [ ] `threadIdx`, `blockIdx`, `blockDim` 의미 파악
- [ ] `cudaMalloc`, `cudaMemcpy` 사용법 숙지
- [ ] 기본적인 에러 디버깅 가능